

What is CSS?

CSS (Cascading Style Sheets) is a stylesheet language used to style and design HTML elements on a web page.

HTML defines the structure and content of a webpage, while CSS controls its appearance and layout.

Ways to Add CSS

CSS can be added to HTML in three ways:

1. Inline CSS

CSS is written directly inside the HTML element using the style attribute for applying a style to a single element.

style="css1: value; css2 : value;"

```
<h1 style="color: blue;">Hello World</h1>
```

2. Internal CSS

CSS is written inside the <style> </style> tag, usually within the <head></head> section for styling a single HTML page.

```
<head>
  <style>
    h1 {
      color: blue;
    }
  </style>
</head>
```

3. External CSS

CSS is written in a separate **.css** file and connected to HTML using the <link> tag.

Best approach for most real-world websites because the same CSS file can be used across multiple HTML pages.

— **index.html (HTML FILE)** —

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

— **style.css (CSS FILE)** —

```
h1 {
  color: blue;
}
```

CSS Selectors

A CSS Selector is used to select the HTML element, that we want to style.

Syntax:

```
selector {  
  property: value;  
}
```

Types of CSS Selectors

1. Element Selector :

Element Selector selects all HTML elements of a specific type or tag.

```
ex.:  p {  
      color: blue;  
}
```

2. ID Selector

ID Selector selects an HTML element using its unique id attribute using (#) symbol.

```
ex.:  #title {  
      color: red;  
}
```

```
<h1 id="title">Apply red color only in this heading tag</h1>
```

3. Class Selector

Class Selector selects one or more HTML elements having the same class name using (.) symbol.

```
ex.:  .text {  
      color: green;  
}
```

```
<h1 class="title">Apply red color only in this heading tag</h1>
```

```
<h3 class="title">Apply red color only in this heading tag</h3>
```

4. Universal Selector

Universal Selector selects all HTML elements on the page using (*) symbol.

```
ex.:  * {  
      margin: 0;  
      padding: 0;  
}
```

5. Grouping Selector

```
h1, h2, p {color: blue; }
```

SOME CSS PROPERTIES

1. CSS Colors

color : red **or** rgb(255, 255, 0) **or** #FEDA00;

background-color

border-color

Color formats: (Name, RGB, RGBA, HEX)

4. CSS Backgrounds

Background properties control the colour and image displayed behind an element.

Properties / Values:

- background-color → red or #ff0000 or rgb(255, 0, 0)
- background-image → url("image.jpg") or linear-gradient(...)
- background-repeat → repeat or no-repeat or repeat-x or repeat-y
- background-position → left or center or right or top or bottom or 50% 50%
- background-size → auto or cover or contain or 100% 100%
- background-attachment → scroll or fixed or local
- background → Shorthand for multiple background properties

7. CSS Text

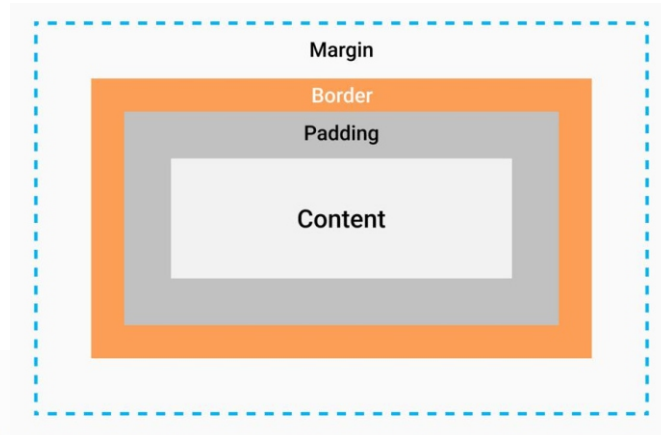
Text properties control alignment, decoration, transformation and spacing.

Properties / Values:

- text-align → left or center or right or justify
- text-decoration → none or underline or overline or line-through
- text-transform → none or uppercase or lowercase or capitalize
- text-indent → 20px or 2em
- line-height → normal or 1.5 or 24px
- letter-spacing → normal or 2px or 0.1em
- word-spacing → normal or 5px or 0.2em
- text-shadow → 2px 2px 5px gray

Box Model :

Every HTML element is treated as a rectangular box with these four parts – content, padding, border, and margin. The Box Model describes how an element's total size is formed

**5. CSS Borders**

Borders create a visible boundary around an element.

Properties / Values:

- `border-style` → `solid` or `dashed` or `dotted` or `double` or `none`
- `border-width` → `1px` or `2px` or `thin` or `medium` or `thick`
- `border-color` → `red` or `#ff0000` or `rgb(...)`
- `border-top` → `2px solid red`
- `border-right` → `2px solid red`
- `border-bottom` → `2px solid red`
- `border-left` → `2px solid red`
- `border-radius` → `10px` or `50%`
- `border` → `1px solid black`

9. CSS Margin & Padding

Margin creates space outside an element, while **padding** creates space inside it.

Properties / Values:

- `margin` → `20px` or `10px 20px` or `10px auto`
- `margin-top` → `20px` or `2rem`
- `margin-right` → `20px` or `2rem`
- `margin-bottom` → `20px` or `2rem`
- `margin-left` → `20px` or `2rem`
- `padding` → `20px` or `10px 20px`
- `padding-top` → `20px` or `2rem`
- `padding-right` → `20px` or `2rem`
- `padding-bottom` → `20px` or `2rem`
- `padding-left` → `20px` or `2rem`

Display & Visibility :

Display controls how an element behaves and occupies space in the layout.

display : block or inline or inline-block or none;

Visibility: visibility controls whether an element is visible or hidden without changing its layout space.

visibility : visible or hidden;

Overflow controls content that goes outside an element's defined area.

overflow : visible or hidden or scroll or auto

overflow-x : visible or hidden or scroll or auto

overflow-y : visible or hidden or scroll or auto

CSS Position

Position properties control where an element is placed relative to its normal position or containing element.

position : static or relative or absolute or fixed or sticky

Note : relative will be treated as parent div and absolute will be treated as child of that relative div

top : 10px or 2rem or 10%

right : 10px or 2rem or 10%

bottom : 10px or 2rem or 10%

left : 10px or 2rem or 10%

z-index : 1 or 10 or 100

Note : z-index is responsible for layers arrangement (top & bottom).

CSS Flexbox

Flexbox is a CSS layout system used to arrange elements inside a container in a row or column. It makes alignment, spacing and responsive layouts easier.

```
<div class="container">  
  <div class="card">Item 1</div>  
  <div class="card">Item 2</div>  
  <div class="card">Item 3</div>  
</div>
```

CSS Flexbox

```
.container{
  display: flex;
  flex-direction: row or row-reverse or column or column-reverse;
  justify-content: value-below;
  align-items: value-below;
  gap: 20px; or row-gap: 20px; or column-gap: 30px;
  flex-wrap: wrap or nowrap or wrap-reverse;
}
```

flex-direction decides the main direction in which flex items are arranged.

row	—	Left to right
row-reverse	—	Right to left
column	—	Top to bottom
column-reverse	—	Bottom to top

justify-content controls how flex items are distributed along the main axis.

flex-start	—	Items start from the beginning
center	—	Items stay in the centre
flex-end	—	Items move to the end
space-between	—	Equal space between items
space-around	—	Space around each item
space-evenly	—	Equal space between and around items

align-items controls the alignment of flex items along the cross axis.

stretch	—	Items stretch by default
flex-start	—	Items move to the start
center	—	Items move to the centre
flex-end	—	Items move to the end

gap is usually cleaner than adding margins to every child just to create spacing.

flex-wrap — By default, flex items try to stay on one line. flex-wrap controls whether they are allowed to move to another line.

nowrap	—	All items stay on one line
wrap	—	Items move to the next line when required
wrap-reverse	—	Items wrap in the opposite cross-axis direction

Note — This property is applied to the **child**.

```
.card { flex-grow: 0 or 1 or 2;}
```

flex-grow controls how much a flex item can grow and take available extra space.